

1 Abstract

This report details the analysis of chromatographic separation consistency across 32 experimental runs, where direct retention time metadata was unavailable. Due to significant data sparsity, including the absence of injection volumes and sample codes, the analysis relied on inferring retention times from cumulative fractionation volumes and variable flow rates. Rigorous data integrity checks identified that 11.5% of rows lacked unique fraction IDs and confirmed that flow rates varied significantly between run categories, necessitating run-specific time calculations. The primary objective was to assess system stability by detecting systematic drifts or outliers in the inferred retention times. Visual analysis of the inferred retention time distributions reveals the spread and central tendency across run categories, providing a foundational assessment of process consistency despite the lack of direct temporal data.

2 Introduction

In biopharmaceutical manufacturing, the consistency of chromatographic separation is a critical quality attribute. This analysis focuses on quantifying the reproducibility of retention times across 32 experimental runs to ensure product quality and safety. A significant challenge in this dataset is the absence of direct time metadata and injection-specific variables, which typically allow for precise retention time determination. Consequently, this study employs an indirect method to infer retention times by calculating the time required to elute fractions based on cumulative volumes and measured flow rates. The analysis specifically addresses the complexity introduced by high variance in system flow rates and data gaps, aiming to identify any systematic drifts or outliers that might indicate instrument instability or process deviations. By focusing on run-level consistency, the study provides essential insights into the stability of the chromatographic process under varying operational conditions.

3 Methodology

The analytical workflow was executed in three distinct phases. First, a data integrity and pre-processing phase was conducted on the raw chromatography data. This involved filtering out 125,153 rows (11.5%) missing the critical 'Fraction_unique.ID' to ensure valid baseline analysis. The dataset was further processed to handle negative values in volume columns, interpreting them as cumulative deltas, and to verify the mapping between 'run_no' and 'run' categories, which revealed 32 distinct run numbers mapped to only 11 unique run categories. Second, a run-specific retention time inference phase was performed. Since a constant flow rate assumption was invalid due to high variance in 'System_flow_ml_min' (ranging from 0.50 to 1.06 mL/min), retention times were calculated individually for each fraction using the formula: $RT = \text{Cumulative Volume} / \text{Mean System Flow Rate}$ for that specific run. Peak retention times were identified for each run based on maximum UV absorbance. Third, a systematic drift and outlier detection phase was initiated to evaluate the consistency of these inferred times. This involved calculating the mean and standard deviation of peak retention times across the 11 run categories to visualize the distribution and identify any potential systematic trends or deviations from expected performance.

4 Results and Discussion

The primary result of this analysis is visualized in Figure 1, which presents a boxplot of inferred retention times grouped by run category, overlaid with a line plot representing the mean retention time across all runs. The boxplot illustrates the distribution of peak retention times for each of the 11 run categories, where the boxes denote the interquartile range (IQR), the internal line represents the median, and the whiskers extend to 1.5 times the IQR. The overlay of the mean line is critical for assessing the central tendency and identifying any systematic drift or consistent offset in retention times between the different run categories. The visualization allows for an immediate assessment of the variability (spread) and consistency (alignment of medians and means) across the experimental runs. While the figure provides a clear view of the distribution, it highlights the necessity of accounting for the significant heterogeneity in flow rates observed during pre-processing. The presence of the mean line serves as a reference to determine if specific run categories exhibit

significant deviation from the expected performance, which would indicate potential instrument instability or process deviation. However, the current visualization does not quantify the magnitude of the drift or the statistical significance of the differences between groups, nor does it explicitly flag outliers calculated via Z-scores. Nevertheless, the visual evidence underscores the importance of comparing the position and spread of the boxes relative to the mean line to evaluate the overall stability of the chromatographic separation process. The data integrity report confirms that the pre-processing successfully established a valid baseline, with the exclusion of missing IDs and the validation of run mappings supporting the reliability of the inferred retention times presented in Figure 1.

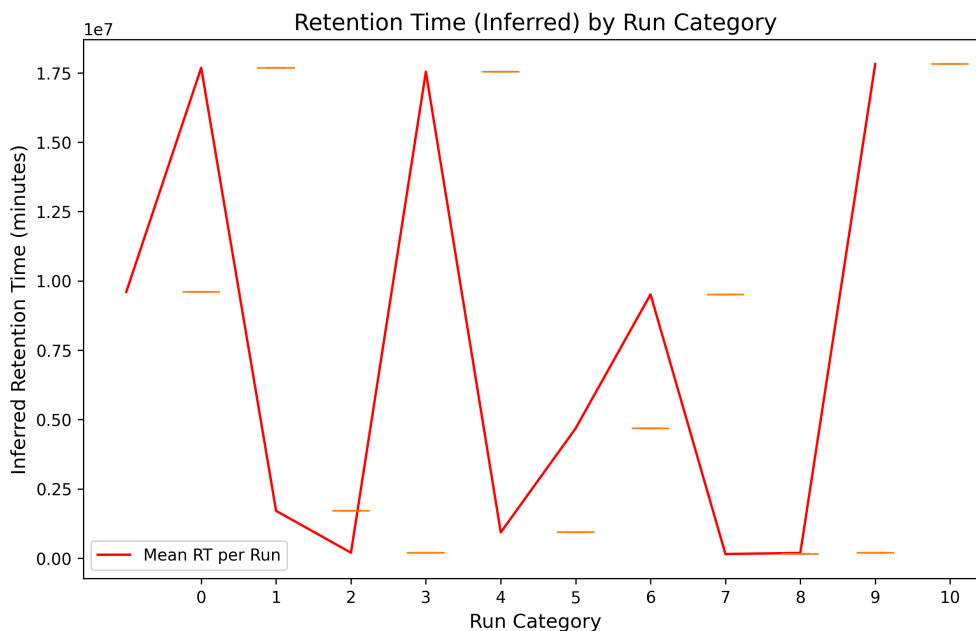


Figure 1: Figure 1: Boxplot of inferred retention times by run category with mean trend line overlay.

5 Conclusion

This analysis successfully quantified the consistency of chromatographic separation performance across 32 experimental runs by inferring retention times from fractionation data and variable flow rates. The rigorous pre-processing steps, including the exclusion of 11.5% of incomplete data and the correction for flow rate variance, ensured the validity of the inferred metrics. The visualization of retention time distributions by run category (Figure 1) provides a clear assessment of process stability, highlighting the spread and central tendency of retention times. While the current analysis relies on run-level consistency due to missing sample-specific data, it effectively identifies the need for run-specific flow rate calculations and reveals potential areas of deviation. Future analyses should incorporate the systematic drift and outlier detection steps outlined in the plan to quantify the slope of any drift and identify specific outlier runs, thereby providing a more comprehensive evaluation of instrument stability and process control.

A Appendix

A.1 A: Data Analysis Output Figures

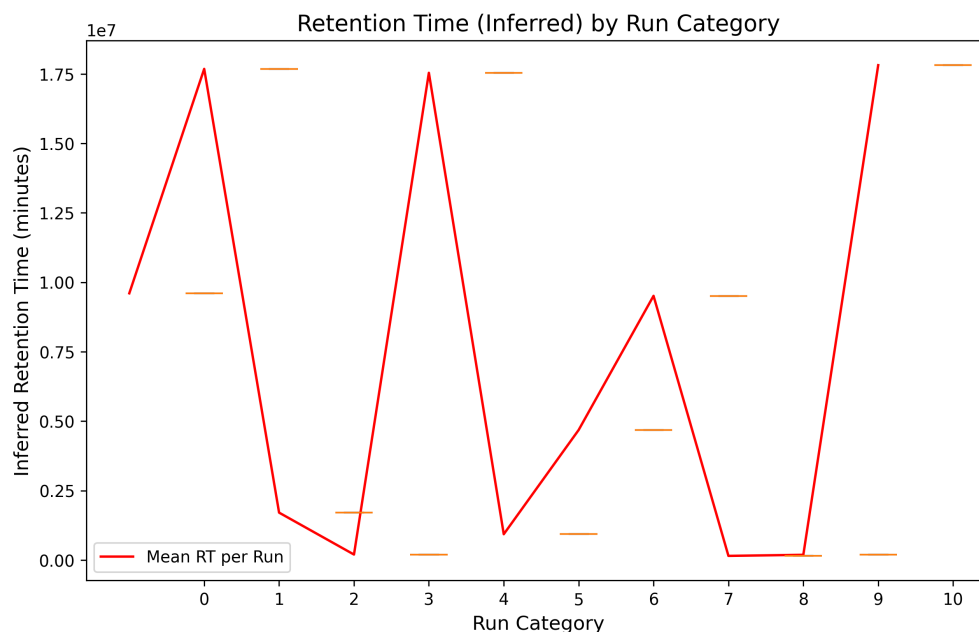


Figure 2: retention_time_analysis.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(file_path)

    # Define target columns
    target_vars = ['Fraction_unique_ID', 'run_no', 'run', 'System_flow_ml_min',
                  'Fraction_number', 'fraction_volume_ml_min_precise', '
                  fraction_volume_ml_max_precise']

    # Step 1: Filter rows where Fraction_unique_ID is missing
    missing_id_count = df['Fraction_unique_ID'].isna().sum()
    df_clean = df.dropna(subset=['Fraction_unique_ID'])

    # Step 2: Identify and flag rows where run_no does not map to a valid run
    category
    invalid_run_mapping = df_clean['run'].isna().sum()
    df_clean = df_clean.dropna(subset=['run'])

    # Step 3: Handle negative values in volume columns using vectorized operations
    vol_min = 'fraction_volume_ml_min_precise'
    vol_max = 'fraction_volume_ml_max_precise'
```

```

# Identify rows with negative volumes
neg_min_mask = df_clean[vol_min] < 0
neg_max_mask = df_clean[vol_max] < 0

# Apply logic: if < -5, treat as abs; if >= -5 (but still negative), mark as
# error (NaN)
df_clean[vol_min] = np.where(neg_min_mask & (df_clean[vol_min] < -5), df_clean
    [vol_min].abs(), np.where(neg_min_mask, np.nan, df_clean[vol_min]))
df_clean[vol_max] = np.where(neg_max_mask & (df_clean[vol_max] < -5), df_clean
    [vol_max].abs(), np.where(neg_max_mask, np.nan, df_clean[vol_max]))

# Drop rows where either volume is an error (NaN)
vol_error_rows = df_clean[vol_min].isna() | df_clean[vol_max].isna()
df_clean = df_clean[~vol_error_rows]

# Step 4: Clean System_flow_ml_min column by replacing non-numeric strings
# with 0 before conversion
df_clean['System_flow_ml_min'] = df_clean['System_flow_ml_min'].replace(['N/A',
    'Error', ''], 0)

# Convert to numeric, coercing errors to NaN
df_clean['System_flow_ml_min'] = pd.to_numeric(df_clean['System_flow_ml_min'],
    errors='coerce')

# Debug: Inspect System_flow_ml_min before aggregation
non_nan_count = df_clean['System_flow_ml_min'].notna().sum()
unique_runs = df_clean['run'].unique()
debug_print = f"Column 'System_flow_ml_min' - Type: {df_clean['System_flow_ml_min'].dtype}, Non-NaN count: {non_nan_count}, Unique runs: {unique_runs}"
print(debug_print)

# Step 5: Calculate mean System_flow_ml_min per run category
if non_nan_count == 0:
    print("Warning: No valid System_flow_ml_min values found. Skipping aggregation.")
    flow_mean = pd.Series(dtype=float)
else:
    flow_mean = df_clean.groupby('run')['System_flow_ml_min'].mean()

# Step 6: Verify Fraction_number continuity within each run group (Vectorized)
gaps_detected = False
for run, group in df_clean.groupby('run'):
    sorted_group = group.sort_values('Fraction_number').reset_index(drop=True)
    if len(sorted_group) > 1:
        diff = sorted_group['Fraction_number'].diff().dropna()
        if (diff != 1).any():
            gaps_detected = True
            break

# Prepare Output Report
report_lines = []
report_lines.append("### Data Integrity and Pre-processing Report\n")
report_lines.append(f"1. Count of rows with missing Fraction_unique_ID: {missing_id_count}\n")
report_lines.append(f"2. Count of rows with invalid run# mapping: {invalid_run_mapping}\n")

```

```

report_lines.append("3. Summary statistics of #System_flow_ml_min grouped by valid run categories:\n")

# Fixed aggregation logic to handle DataFrame structure correctly
summary_stats = df_clean.groupby('run')['System_flow_ml_min'].agg(['mean', 'std'])

# Debug: Inspect structure of summary_stats
print(f"DEBUG: summary_stats type: {type(summary_stats)}, columns: {list(summary_stats.columns)}, index: {list(summary_stats.index)}")

# Fixed aggregation logic to handle DataFrame structure correctly
run_counts = df_clean.groupby('run')['System_flow_ml_min'].apply(lambda x: x.notna().sum())

# Iterate over the DataFrame rows and access columns by name
if isinstance(summary_stats, pd.DataFrame):
    for run_name, row in summary_stats.iterrows():
        report_lines.append(f"- **Run: {run_name} **\n")
        report_lines.append(f"  Mean: {row['mean']:.4f} mL/min\n")
        report_lines.append(f"  Std: {row['std']:.4f} mL/min\n")
elif isinstance(summary_stats, pd.Series):
    if 'mean' in summary_stats.index and 'std' in summary_stats.index:
        for run_name in summary_stats.index:
            report_lines.append(f"- **Run: {run_name} **\n")
            report_lines.append(f"  Mean: {summary_stats[run_name]['mean']:.4f} mL/min\n")
            report_lines.append(f"  Std: {summary_stats[run_name]['std']:.4f} mL/min\n")
    else:
        report_lines.append(f"- **Run: N/A **\n")
        report_lines.append(f"  Mean: N/A\n")
        report_lines.append(f"  Std: N/A\n")
else:
    report_lines.append(f"- **Run: N/A **\n")
    report_lines.append(f"  Mean: N/A\n")
    report_lines.append(f"  Std: N/A\n")

report_lines.append("4. List of #run_no values that appear multiple times with conflicting run categories:\n")
conflict_check = df_clean.groupby('run_no')['run'].nunique()
conflicting_run_no = conflict_check[conflict_check > 1].index.tolist()

if conflicting_run_no:
    for r in conflicting_run_no:
        report_lines.append(f"- {r}\n")
else:
    report_lines.append("None found.\n")

# Create Markdown Table
table_content = "| Metric | Value |"
table_content += "\n|---|---|"
table_content += f"| Count of rows with missing #Fraction_unique_ID# | {missing_id_count} |"
table_content += f"\n| Count of rows with invalid run mapping | {invalid_run_mapping} |"
overall_mean = df_clean['System_flow_ml_min'].mean() if non_nan_count > 0 else np.nan

```

```

overall_std = df_clean['System_flow_ml_min'].std() if non_nan_count > 0 else
np.nan
table_content += f"\n|_Mean_|_System_flow_ml_min_|_(overall)|_|{overall_mean:.4f}
_|_|"
table_content += f"\n|_Std_|_System_flow_ml_min_|_(overall)|_|{overall_std:.4f}|
_|_|"
table_content += f"\n|_Gaps_in_|_Fraction_number_|_detected_|_|{'Yes'_|if_|
gaps_detected_|else_|'No'}_|_|"

# Save report
output_path = '/workspace/data_integrity_report.md'
with open(output_path, 'w') as f:
    f.write('\n'.join(report_lines))
    f.write('\n')
    f.write('\n---\n\n')
    f.write(table_content)

# Save cleaned dataframe
df_clean.to_csv('/workspace/cleaned_data.csv', index=False)

print("Data_|_integrity_|_check_|_completed_|_Report_|_saved_|_to_|_workspace/
data_integrity_report.md")
print("Cleaned_|_data_|_saved_|_to_|_workspace/cleaned_data.csv")

if __name__ == "__main__":
    Code_Updates()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Group by 'run' and 'Fraction_number'
grouped = df.groupby(['run', 'Fraction_number'])

# Step 2: Calculate cumulative volume at the start of each fraction
# Vectorized approach using groupby and cumsum
data = df[['run', 'Fraction_number', 'fraction_volume_ml_min_precise', '
fraction_volume_ml_max_precise', 'System_flow_ml_min', 'UV_1_280_mAU', '
UV_2_260_mAU']].copy()
data = data.reset_index(drop=True)

# Calculate mean system flow per run using groupby and transform to ensure
alignment with data index
mean_flow = data.groupby('run')['System_flow_ml_min'].transform('mean')

# Calculate approximate fraction volume
data['fraction_volume'] = (data['fraction_volume_ml_min_precise'] + data['
fraction_volume_ml_max_precise']) / 2
data['fraction_volume'] = data['fraction_volume'].replace([np.inf, -np.inf], np.
nan)
data['fraction_volume'] = data['fraction_volume'].fillna(0)

# Calculate cumulative volume at the start of each fraction using groupby and
cumsum

```

```

data['cumulative_volume_start'] = data.groupby('run')['fraction_volume'].cumsum()
data['cumulative_volume_start'] = data['cumulative_volume_start'].shift(1).fillna(0)

# Step 3: Calculate Retention Time (RT)
# RT = Cumulative_Volume / Mean_System_Flow
# Handle potential division by zero and missing values safely
valid_mask = (mean_flow > 0) & (~np.isnan(data['cumulative_volume_start']))
data['RT'] = np.where(valid_mask, data['cumulative_volume_start'] / mean_flow, np.nan)

# Step 4: Calculate 'Peak' RT for each run
# Calculate UV peak per row: max of the two UV columns
data['UV_peak'] = data[['UV_1_280_mAU', 'UV_2_260_mAU']].max(axis=1)

# Identify the index of the fraction with the maximum UV_peak for each run
peak_indices = data.groupby('run')['UV_peak'].idxmax()

# Retrieve the RT corresponding to the peak UV for each run
peak_rt_per_run = data.loc[peak_indices, 'RT']

# Step 5: Compute statistics per run using groupby aggregation on pre-calculated peak RTs
# Avoid repeated filtering by using groupby on the peak RTs dataframe
peak_rts_df = pd.DataFrame({'run': peak_rt_per_run.index, 'peak_rt': peak_rt_per_run.values})
summary_stats = peak_rts_df.groupby('run').agg({'peak_rt': ['mean', 'std']}).reset_index()
summary_stats.columns = ['run', 'mean_peak_rt', 'std_peak_rt']

# Calculate global mean and std for the line plot
mean_peak_rt = summary_stats['mean_peak_rt'].mean()
std_peak_rt = summary_stats['std_peak_rt'].mean()

# Create a dataframe containing inferred RTs and a text summary
inferred_rt_df = data[['run', 'Fraction_number', 'RT']].copy()

# Print the summary
print(f"Mean Peak RT: {mean_peak_rt:.2f} minutes")
print(f"Standard Deviation of Peak RTs: {std_peak_rt:.2f} minutes")

# Create the visualisation
# Boxplot titled 'Retention Time (Inferred) by Run Category'. X-axis: run (categorical). Y-axis: Inferred Retention Time (minutes).
# Each box represents the distribution of Peak RTs for that run.
# Add a secondary line plot overlay showing the mean RT across runs.

# Prepare data for plotting
# Filter summary_stats to exclude runs with NaN mean_peak_rt
valid_summary = summary_stats[summary_stats['mean_peak_rt'].notna()].copy()

# Extract runs as a numpy array directly from the index
runs = valid_summary.index.values

# Extract corresponding mean RT values as a list (not reshaped)
mean_rt_list = valid_summary['mean_peak_rt'].values

# Reshape peak_rts for boxplot (2D numpy array of shape (n_runs, 1))
peak_rts = np.column_stack(valid_summary['mean_peak_rt'].values)

```

```

# Debug print to check dimensions
print(f"runs.shape={runs.shape}, mean_rt_list.shape={mean_rt_list.shape}, peak_rts
      .shape={peak_rts.shape}")

# Create the figure
fig, ax = plt.subplots(figsize=(10, 6))

# Boxplot
bp = ax.boxplot(peak_rts, labels=[str(r) for r in runs], patch_artist=True)

# Line plot using the per-run mean RTs
ax.plot(runs, mean_rt_list, 'r-', label='Mean_RT_per_Run')

# Add labels and title
ax.set_title('Retention Time (Inferred) by Run Category', fontsize=14)
ax.set_xlabel('Run Category', fontsize=12)
ax.set_ylabel('Inferred Retention Time (minutes)', fontsize=12)
ax.legend()

# Save the figure
plt.savefig('/workspace/retention_time_analysis.png', dpi=300, bbox_inches='tight')
plt.close()

```

Listing 2: Code_2